

Lab Title:- SYN FLOOD ATTACK INVESTIGATION USING TSHARK

Student Name:- BADAMASI MUNTAKA MUHAMMAD

Student ID:- 2025FWSD11563

Course Name:- BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

Instructor Name:- COMMANDANT AMINU IDRIS

Date Of Submission:- 11/09/2026

Executive summary

The lab covers a practical analysis on establishing and capturing SYN FLOOD ATTACK traffic by executing a python script and then analyse the captured traffic using different tshark command option and then compare it with a normal baseline handshake.

Lab Objectives

The objectives of the lab is to establish & capture a SYN FLOOD ATTACK traffic and then perform some analysis based on the captured traffic using different tshark command option.

Tools and Resources Used

- Kali linux:- A penetration testing platform used for security assessments.

Kali linux tools

- Apache2:- this is the Apache HyperText Transfer Protocol (HTTP) server program that is designed to be run as a standalone daemon process.
 - Wireshark:- is a GUI network protocol analyzer that lets you interactively browse packet data from a live network or from a previously saved capture file.
 - Python3-scapy:- is a powerful interactive packet manipulation tool, packet generator, network scanner, network discovery, packet sniffer.
-

Methology

- I created a new directory with different folder
- I establish and capture a normal handshake baseline using tshark
- I create a python3 script
- I execute the python3 script to create a SYN FLOOD ATTACK and then capture the traffic generated by the python3 script using tshark.
- I made a copy of the captured traffic and then compare their hash(sha256sum) value

- I analyse the captured traffic based on initial SYN,SYN-ACK response,ACK_reset,SYN_count,unique SYN_source_ports using different tshark command option.
- I compared the captured SYN FLOOD ATTACK traffic with a normal baseline handshake.

Screenshot and Evidence

SCREENSHOT SHOWING THE APACHE SERVICE AND TCP PORT 80

```
(root@kali)~[/home/kali/KIT]
# curl -v http://127.0.0.1
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 43116
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 11 Sep 2026 14:13:48 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 20 Mar 2026 06:45:09 GMT
< ETag: "29cf-64d6f06ad6fbf"
< Accept-Ranges: bytes
< Content-Length: 10703
< Vary: Accept-Encoding
< Content-Type: text/html
```

SCREENSHOT SHOWING THE NORMAL HANDSHAKE IN THE BASELINE CAPTURE

```
(root@kali)~[/home/kali/KIT]
# tshark -r normal_http.pcapng -Y 'tcp.flags.syn=1 || tcp.flags.fin=1' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/normal_handshake_flags.tsv
tee: reports/normal_handshake_flags.tsv: No such file or directory
Running as user "root" and group "root". This could be dangerous.
1      2026-09-11T09:26:27.605401769-0400      127.0.0.1      38128      127.0.0.1      80      0
x0002
2      2026-09-11T09:26:27.605415413-0400      127.0.0.1      80      127.0.0.1      38128
0x0012
8      2026-09-11T09:26:27.608904234-0400      127.0.0.1      38128      127.0.0.1      80      0
x0011
9      2026-09-11T09:26:27.608927870-0400      127.0.0.1      80      127.0.0.1      38128
0x0011
(root@kali)~[/home/kali/KIT]
```

SCREENSHOT SHOWING THE BOUNDED SCAPY SCRIPT OUTPUT

```
(root@kali)-[/home/kali/KIT]
# cat syn_probe_lab.py
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')

(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE INITIAL SYN PACKET LIST

```
Browse the World Wide Web
tshark -r Working/bounded_syn_activity.pcapng -Y 'tcp.flags.syn=1 && tcp.flags.ack=0' -T
fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp
.seq \
| tee Reports/initial_syns.tsv
Running as user "root" and group "root". This could be dangerous.
1      1789133781.501067922    127.0.0.1      10829    127.0.0.1      80          0
4      1789133781.501599396    127.0.0.1      17500    127.0.0.1      80          0
7      1789133781.502047085    127.0.0.1      34645    127.0.0.1      80          0
10     1789133781.502526518    127.0.0.1      2972     127.0.0.1      80          0

(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE SYN-ACK RESPONSE LIST

```
(root@kali)-[/home/kali/KIT]
# tshark -r Working/bounded_syn_activity.pcapng -Y 'tcp.flags.syn=1 && tcp.flags.ack=1' -T
fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp
.ack \
| tee Reports/syn_ack_responses.tsv
Running as user "root" and group "root". This could be dangerous.
2      1789133781.501090493    127.0.0.1      80        127.0.0.1      10829       1
5      1789133781.501609426    127.0.0.1      80        127.0.0.1      17500       1
8      1789133781.502054133    127.0.0.1      80        127.0.0.1      34645       1
11     1789133781.502533608    127.0.0.1      80        127.0.0.1      2972        1

(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE ACK/RST BEHAVIOUR

```
(root@kali)-[/home/kali/KIT]
# tshark -r Working/bounded_syn_activity.pcapng -Y 'tcp.flags.reset=1 || (tcp.flags.ack=1
&& tcp.len=0)' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp
.flags \
| tee Reports/ack_reset_candidates.tsv
Running as user "root" and group "root". This could be dangerous.
2      1789133781.501090493    127.0.0.1      80        127.0.0.1      10829       0x0012
3      1789133781.501129142    127.0.0.1      10829     127.0.0.1      80          0x0004
5      1789133781.501609426    127.0.0.1      80        127.0.0.1      17500       0x0012
6      1789133781.501639627    127.0.0.1      17500     127.0.0.1      80          0x0004
8      1789133781.502054133    127.0.0.1      80        127.0.0.1      34645       0x0012
9      1789133781.502056947    127.0.0.1      34645     127.0.0.1      80          0x0004
11     1789133781.502533608    127.0.0.1      80        127.0.0.1      2972        0x0012
12     1789133781.502536620    127.0.0.1      2972     127.0.0.1      80          0x0004

(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE COUNTS BY SOURCE/DESTINATION

```
(root@kali)-[/home/kali/KIT]
# tshark -r Working/bounded_syn_activity.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T
fields \
-e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
| tee Reports/syn_counts_by_pair.txt
Running as user "root" and group "root". This could be dangerous.
  4 127.0.0.1      127.0.0.1      80
(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE UNIQUE SOURCE PORT

```
(root@kali)-[/home/kali/KIT]
# tshark -r Working/bounded_syn_activity.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T
fields -e tcp.srcport \
| sort -n | uniq | tee Reports/unique_syn_source_ports.txt
Running as user "root" and group "root". This could be dangerous.
2972
10829
17500
34645
(root@kali)-[/home/kali/KIT]
```

SCREENSHOT SHOWING THE EVIDENCE HASH

```
(root@kali)-[/home/kali/KIT]
# sha256sum bounded_syn_activity.pcapng Working/bounded_syn_activity.pcapng | tee Reports/bounded_capture_hashes.txt
5cf6bb73339b9ebb78f9486cc7eb8c4ed1e54afddd3de9b0f7bdae1c272143b6  bounded_syn_activity.pcapng
5cf6bb73339b9ebb78f9486cc7eb8c4ed1e54afddd3de9b0f7bdae1c272143b6  Working/bounded_syn_activity.pcapng
(root@kali)-[/home/kali/KIT]
#
```

TABLE SHOWING NORMAL VERSUS SUSPICIOUS COMPARISON TABLE

INDICATOR	NORMAL HTTP SESSION	BOUNDED SYN ACTIVITY	FORENSIC MEANING
Initial SYN Count	The initial SYN Count in a normal HTTP session is 1	The initial SYN count in a bounded SYN activity is 4	A normal session begins with one connection attempt, while four SYN packets indicate multiple connection attempts.

SYN-ACK Count	The SYN-ACK Count in normal HTTP session is 1	The SYN-ACK Count in bounded SYN activity is 4	The normal session received only one SYN-ACK while the bounded activity received four SYN-ACK.
Completed handshakes	The number of completed handshakes in normal HTTP session is 1	The number of completed handshake in bounded SYN activity is 0	The normal session perform the completed TCP three-way handshake, while the bounded SYN activity does not not perform any completed handshakes.
Unique client source ports	The unique client source port in a normal HTTP session is 38128	There are different unique client source ports in a bounded SYN activity which include 2972,10829,17500,34645	A One source port represents a normal connection while four different source ports indicate separate connection attempts.
HTTP request present	The request present in a normal HTTP session is HTTP 1.1	There is no HTTP in bounded SYN activity request	The normal HTTP session proceeded to application-layer HTTP communication, while the bounded activity stopped at the TCP connection stage.
RST packets	The normal HTTP session has a zero(0) RST packet(reset packet)	The bounded SYN activity has four(4) RST packet(reset packet)	No reset behavior was observed in the normal HTTP session while Four RST packets were observed in the bounded activity which means that that the attempted connections were reset rather than completed.
Observed duration	During the establishment of a	During the establishment of a bounded SYN	The normal HTTP session lasted longer

	normal HTTP session the observed duration may last up to 3.53ms	activity the observed session was short which I may last up to 1.47ms	because it completed the handshake and exchanged HTTP data. The bounded activity was very short because the connections were rapidly reset.
--	---	---	---

Conclusion

In conclusion the lab has help me understand how to establish and captured a SYN FLOOD ATTACK and then analyse the captured traffic using different tshark command option.

Reference

➤ ICDFA course Material (SBT-DF203(basic networking skills for digital forensics)- by COMMANDANT AMINU IDRIS

➤ ICDFA Recorded sessions
